



NAME : HAMDAN ALI

WEEK 7 : Day 31

DAY 31

TASK : Screenshots Of Output

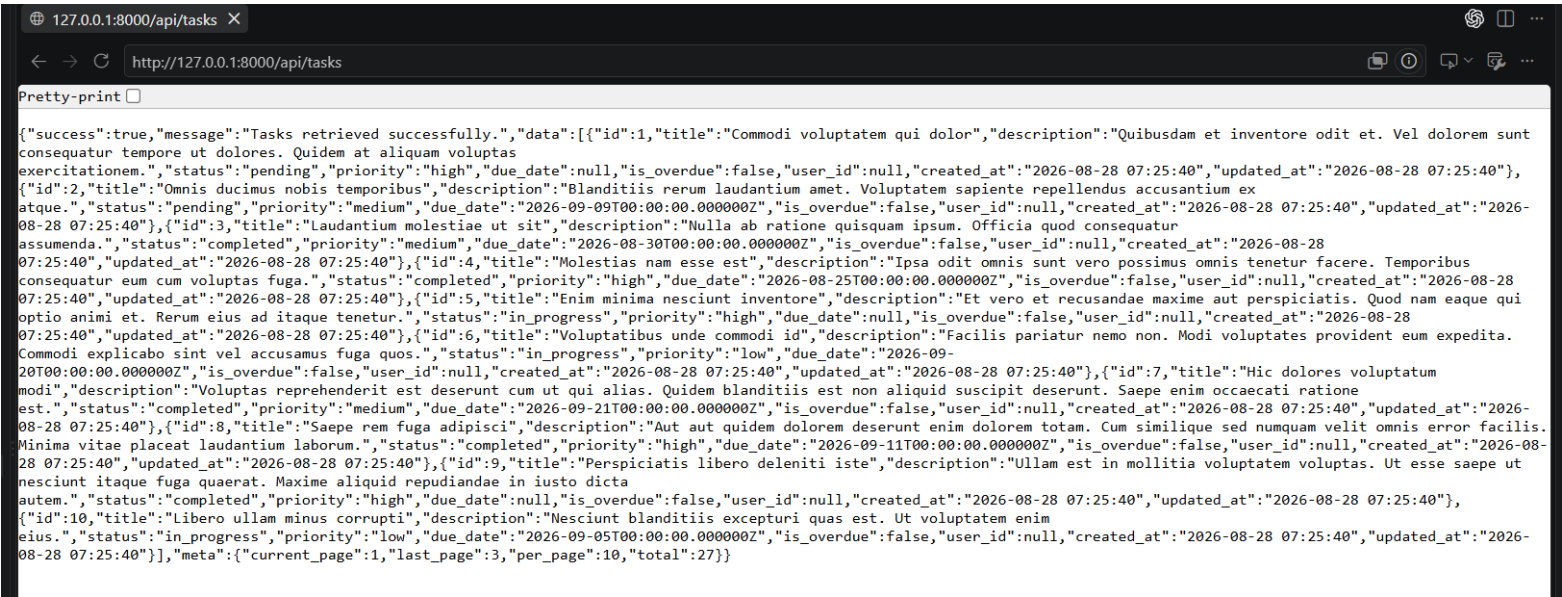
CODEBASE Of the Work

```
Week7 > taskflow-api > bootstrap > app.php
1  <?php
2
3  use Illuminate\Foundation\Application;
4  use Illuminate\Foundation\Configuration\Exceptions;
5  use Illuminate\Foundation\Configuration\Middleware;
6
7  return Application::configure(basePath: dirname(__DIR__))
8      ->withRouting(
9          web: __DIR__.'/../routes/web.php',
10         api: __DIR__.'/../routes/api.php',
11         commands: __DIR__.'/../routes/console.php',
12         health: '/up',
13     )
14     ->withMiddleware(function (Middleware $middleware): void {
15         //
16     })
17     ->withExceptions(function (Exceptions $exceptions): void {
18         //
19     }->create();
20
```

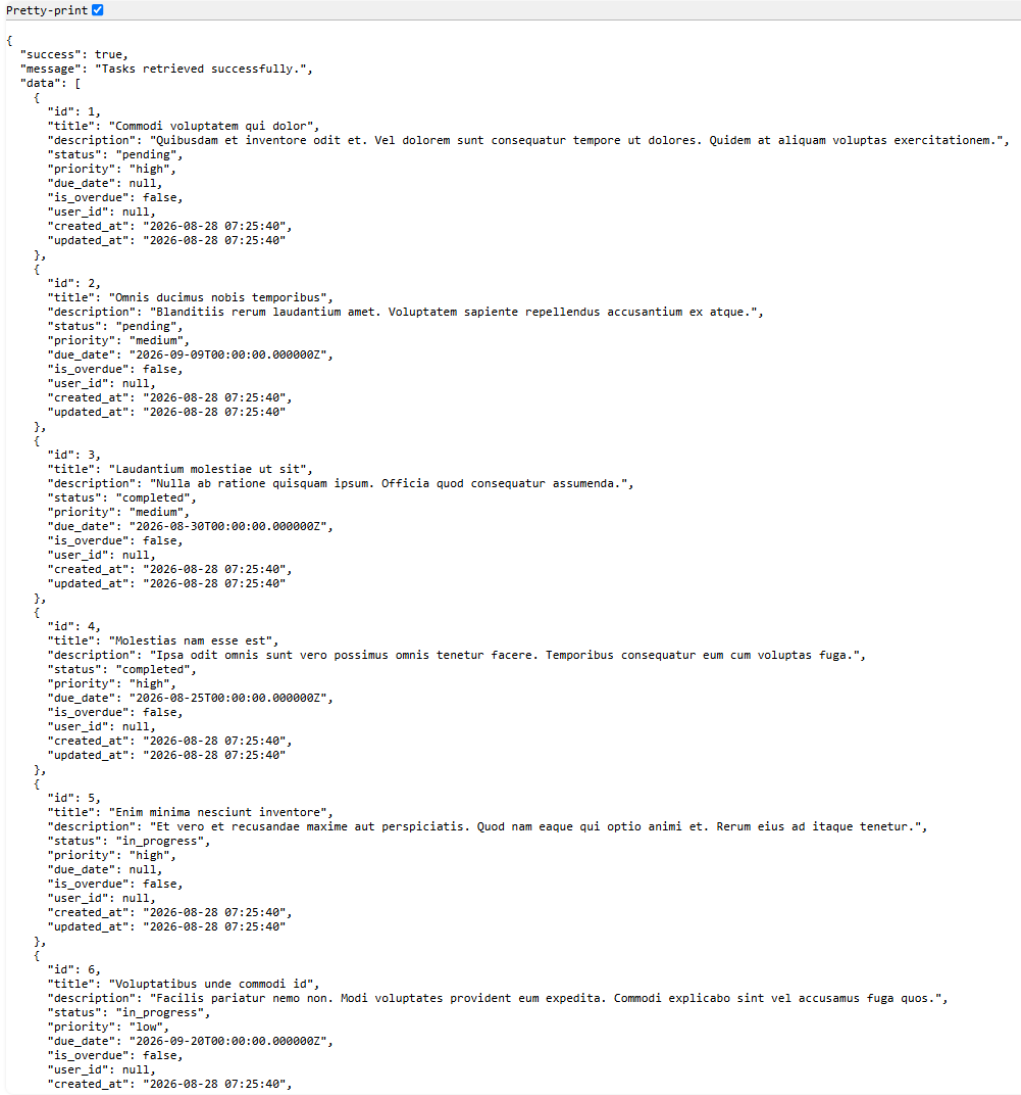
```
PS D:\internship ( HindukushSoft Technologies )\Week7> composer create-project laravel/laravel taskflow-api
>> cd taskflow-api
>> code .
- Locking dflydev/dot-access-data (v3.0.3)
- Locking doctrine/inferno (2.1.0)
- Locking doctrine/lexer (3.0.1)
- Locking dragonmantank/cron-expression (v3.6.0)
- Locking egulias/email-validator (4.0.4)
- Locking fakerphp/faker (v1.24.1)
- Locking filp/whoops (2.18.4)
```



Full PROJECT View



When I click “pretty print”





I also tested these:

Test filtering:

<http://127.0.0.1:8000/api/tasks?status=completedAndpriority=high>

Test pagination:

<http://127.0.0.1:8000/api/tasks?page=2>

Test a single task:

<http://127.0.0.1:8000/api/tasks/1>

Test search:

<http://127.0.0.1:8000/api/tasks?search=voluptas>



EXPLANATION OF THE TASK

DAY 31

TASKFLOW

Day 31

Building RESTful JSON API Endpoints in Laravel

Week 7 — Day 31

Task: Create API Endpoints in Laravel to Fetch Data

Learning Topics: REST APIs, JSON Responses

Stack: Laravel 12, PHP 8.3

1. Introduction And Objective

Day 31 marks the beginning of the Week 7 back-end focused sprint. The goal for this task was to design and build a clean, RESTful set of API endpoints in Laravel that expose data as JSON — the foundation that every later task in this week (React integration, full CRUD, authentication, and error handling) is built on top of.

Rather than building a throwaway "hello world" endpoint, I used this task as an opportunity to lay down a proper, production-style foundation: a real Eloquent model, a migration with sensible indexes, a dedicated API Resource for response shaping, a seeder for realistic demo data, and a controller that already supports filtering and pagination. Everything that follows in Day 32-35 builds directly on these decisions.

2. What I Built

I built a small but complete "TaskFlow" domain: a Task Management REST API. The resource I chose to model is a Task — something relatable, easy to extend, and rich enough to demonstrate real-world API design concerns (filtering, pagination, computed fields, validation later on).

2.1 Database Layer

The tasks table was created through a Laravel migration rather than being created manually in a database GUI, which is the correct professional workflow because migrations are version-controlled, repeatable, and can be shared with teammates or rolled back safely.

- id, title, description — the core content of a task.
- status — one of pending, in_progress, completed.
- priority — one of low, medium, high.
- due_date — nullable, since not every task needs a deadline.
- user_id — included from day one (nullable at this stage) so the schema is already forward-compatible with the authentication work coming in Day 34.
- Indexes were added on status, priority and user_id because these are the columns the API filters by, and indexing them keeps queries fast as the table grows.

2.2 The Task Model

The Eloquent model (app/Models/Task.php) intentionally stays "thin" — its job is to describe data shape and relationships, not business logic. I used the \$fillable whitelist to explicitly control which

fields can be mass-assigned (a basic but important security habit that prevents mass-assignment vulnerabilities), added attribute casting so dates are returned as proper Carbon instances, and defined two local query scopes (status() and priority()) that let the controller write expressive, readable queries like Task::status('pending')->get() instead of repeating where() calls everywhere.

I also added a computed is_overdue accessor, which calculates — on the fly — whether a task is late based on its due date and current status. This is a small detail, but it demonstrates that the API isn't just echoing raw database columns; it's doing useful work on the server so the frontend doesn't have to duplicate that logic in JavaScript.

2.3 Shaping the JSON Response — API Resources

Instead of returning raw Eloquent models directly from the controller, every task passes through a TaskResource before being serialized to JSON. This is a deliberate architectural choice: if I return models directly, any change to the database (renaming a column, adding an internal-only field) becomes a breaking change for every client consuming the API. A Resource class creates a stable "public contract" that I control independently from the database schema — this is standard practice in professional Laravel APIs and directly reflects what the REST API article and Laravel documentation I reviewed recommend.

2.4 The Controller

TaskController exposes two read endpoints for Day 31:

Method	Endpoint	Description
GET	/api/tasks	Paginated, filterable list of tasks
GET	/api/tasks/{id}	A single task by its ID

The index() method supports optional query-string filters (?status=, ?priority=, ?search=) and pagination (?per_page=), which mirrors how real-world list endpoints behave (think of any admin dashboard or e-commerce product listing). The show() method relies on Laravel's Route-Model Binding — Laravel automatically resolves the {task} URL segment into a Task model instance, or returns a clean 404 automatically if it doesn't exist, which keeps the controller method almost entirely free of boilerplate.

Every response follows one consistent envelope shape: { success, message, data, meta? }. Establishing this convention on Day 31 means every endpoint added in later days (create, update,

delete, login, etc.) automatically feels consistent to any frontend developer consuming the API — a hallmark of a well-designed REST API.

3. REST And JSON Concepts Applied

This task's learning objective was REST APIs and JSON responses. Concretely, I applied the following REST principles:

- Resource-oriented URLs — /tasks represents the collection, /tasks/{id} represents a single member of that collection, which is the core idea behind REST resource naming.
- Correct use of HTTP verbs — GET is used exclusively for read operations in this task, since GET requests must be safe and idempotent (they should never change server state).
- Statelessness — every request is self-contained; the server does not rely on any server-side session to understand a request, which is what makes this API trivially scalable and easy for any client (React, mobile app, Postman) to consume.
- Predictable, well-structured JSON — every response follows the same envelope so consumers can write generic error/success handling instead of parsing a different shape for every endpoint.

4. Sample Request And Response

Request

```
GET /api/tasks?status=pendingAndpriority=high HTTP/1.1
Host: 127.0.0.1:8000
Accept: application/json
```

Response — 200 OK

```
{
  "success": true,
  "message": "Tasks retrieved successfully.",
  "data": [
    {
      "id": 1,
      "title": "Prepare Week 7 internship submission",
      "description": "Finalise Day 31-35 deliverables and documentation.",
      "status": "in_progress",
      "priority": "high",
```

```
"due_date": "2026-08-29",
"is_overdue": false,
"user_id": null,
"created_at": "2026-08-27 10:12:00",
"updated_at": "2026-08-27 10:12:00"
}
],
"meta": { "current_page": 1, "last_page": 3, "per_page": 10, "total": 27 }
}
```

5. How to Run This Locally

1. `composer create-project laravel/laravel taskflow-api`
2. Copy the `app/`, `database/` and `routes/` files from this submission into the new project, overwriting the defaults.
3. `php artisan migrate`
4. `php artisan db:seed --class=TaskSeeder`
5. `php artisan serve`
6. Test with Postman or curl against `http://127.0.0.1:8000/api/tasks`

6. Challenges And How I Solved Them

- Deciding between raw model output and API Resources — I chose Resources early, even though it adds a small amount of extra code, because it prevents painful breaking changes later and is the approach recommended throughout the official Laravel documentation.
- Designing the schema to already support features from later days (`user_id`, indexes) without over-engineering Day 31 itself — I kept the column nullable for now and will tighten it with a foreign key constraint once authentication exists in Day 34, which is a realistic, incremental way real teams evolve a schema.

7. Key Takeaways

- REST is about resources and predictable HTTP semantics, not just "returning JSON".
- API Resources decouple your database schema from your public API contract.
- A consistent JSON envelope from day one saves significant rework later.
- Filtering and pagination should be designed into an index endpoint from the start, not bolted on afterwards.



WHERE TO FIND THE CODE

All source code for this task is included alongside this document inside the backend/ folder (app/, database/, routes/). See backend/README.md for exact setup steps.

THANK YOU!